# DS-604 Project Documentation

## Formula 1 World Championship (1950–2024): Database Design and Performance Analysis

### Milestone 5: Saving Queries as Views

**Student ID:**
**202618052**
**202618016**
**202618013**

# Contents


| No. | Name | Page No. |
|---|---|---|
| 1. | Overview | 3 |
| 2. | Summary of Views | 3 |
| 3. | SQL Code of the Views | 4 |
| 4. | Checking the Views | 9 |

# 1. Overview

This document is the fifth milestone deliverable for the DS604 group project. Ten of the analytical queries from Milestone 4 are saved as **views** in the `DBMS_project` schema of the PostgreSQL database. A view stores the query under a name, so its result can be retrieved at any time with a simple `SELECT` instead of rewriting the full query.

**Target Host:** 10.100.71.21

**Target Database / Schema:** 202618052 / DBMS_project

**Source Data:** Formula 1 World Championship (1950–2024)

**Due Date:** 6 November 2026

Each view is created with `CREATE OR REPLACE VIEW`, so the statements can be run again without errors. `LIMIT` clauses used in Milestone 4 are not included, so every view returns its complete result. In the uploaded `results` table, missing values of `position` and `rank` are stored as the text `\N`; the views convert them with `NULLIF(position, '\N')::int` and select the fastest lap with `rank = '1'`.

# 2. Summary of Views

| No. | View Name | Based on | Description |
|---|---|---|---|
| 1 | `v_driver_career_wins` | Query 1 – Career wins | Career wins, podiums and win percentage of every driver |
| 2 | `v_constructor_championships` | Query 2 – Constructor dominance | Constructors' Championship titles and title years of every constructor |
| 3 | `v_driver_career_points` | Query 3 – All-time points leaders | Career championship points, seasons and titles of every driver |
| 4 | `v_pitstop_efficiency` | Query 4 – Pit-stop efficiency | Pit-stop performance of each constructor across seasons |
| 5 | `v_dnf_analysis` | Query 9 – DNF / retirement analysis | Race retirements of each constructor grouped by cause |
| 6 | `v_season_points_progression_2021` | Query 10 – Season points progression | Race-by-race championship progression of the 2021 top 5 drivers |
| 7 | `v_home_race_advantage` | Query 12 – Home-race advantage | Home Grand Prix finishing position compared with the driver's other races |
| 8 | `v_decade_team_dominance` | Query 13 – Decade-wise team dominance | Top 3 constructors of each decade by share of points |
| 9 | `v_grid_vs_points` | Query 14 – Grid position vs. points scored | Points, wins and podiums by starting grid position (2010 onward) |
| 10 | `v_sprint_vs_gp` | Query 15 – Sprint vs. Grand Prix performance | Sprint result compared with the Grand Prix result on the same weekend |

# 3. SQL Code of the Views

The statements below are run on the server after setting the schema:

```
SET search_path TO "DBMS_project";
```

## 3.1 View 1: v_driver_career_wins

**Based on:** Query 1 – Career wins | **Purpose:** Career wins, podiums and win percentage of every driver

```
CREATE OR REPLACE VIEW "DBMS_project".v_driver_career_wins AS
SELECT d.forename || ' ' || d.surname                        AS driver,
       d.nationality,
       COUNT(*) FILTER (WHERE NULLIF(r.position, '\N')::int = 1)                AS wins,
       COUNT(*) FILTER (WHERE NULLIF(r.position, '\N')::int <= 3)               AS podiums,
       COUNT(*)                                              AS races_entered,
       ROUND(100.0 * COUNT(*) FILTER (WHERE NULLIF(r.position, '\N')::int = 1) / COUNT(*), 1) AS win_pct
FROM results r
JOIN drivers d ON d."driverId" = r."driverId"
GROUP BY d."driverId", d.forename, d.surname, d.nationality
ORDER BY wins DESC, podiums DESC;
```

## 3.2 View 2: v_constructor_championships

**Based on:** Query 2 – Constructor dominance | **Purpose:** Constructors' Championship titles and title years of every constructor

```
CREATE OR REPLACE VIEW "DBMS_project".v_constructor_championships AS
WITH season_final_race AS (          -- last race of every season
    SELECT year, MAX(round) AS last_round
    FROM races
    GROUP BY year
),
champions AS (                       -- constructor in P1 after the final race
    SELECT ra.year, cs."constructorId"
    FROM constructor_standings cs
    JOIN races ra             ON ra."raceId" = cs."raceId"
    JOIN season_final_race sf ON sf.year = ra.year AND sf.last_round = ra.round
    WHERE cs.position = 1
)
SELECT c.name                                            AS constructor,
       COUNT(*)                                          AS championships,
       MIN(ch.year)                                      AS first_title,
       MAX(ch.year)                                      AS latest_title,
       STRING_AGG(ch.year::text, ', ' ORDER BY ch.year)  AS title_years
FROM champions ch
JOIN constructors c ON c."constructorId" = ch."constructorId"
GROUP BY c.name
ORDER BY championships DESC, first_title;
```

## 3.3 View 3: v_driver_career_points

**Based on:** Query 3 – All-time points leaders | **Purpose:** Career championship points, seasons and titles of every driver

```
CREATE OR REPLACE VIEW "DBMS_project".v_driver_career_points AS
WITH season_final_race AS (
    SELECT year, MAX(round) AS last_round
    FROM races
    GROUP BY year
),
season_end AS (                      -- each driver's final standing in each season
    SELECT ds."driverId", ra.year, ds.points, ds.position
    FROM driver_standings ds
    JOIN races ra             ON ra."raceId" = ds."raceId"
    JOIN season_final_race sf ON sf.year = ra.year AND sf.last_round = ra.round
)
SELECT d.forename || ' ' || d.surname           AS driver,
       SUM(se.points)                          AS career_points,
       COUNT(*)                                AS seasons,
       COUNT(*) FILTER (WHERE se.position = 1)   AS world_titles,
       MAX(se.points)                          AS best_season_points,
       ROUND((SUM(se.points) / COUNT(*))::numeric, 1)       AS points_per_season
FROM season_end se
JOIN drivers d ON d."driverId" = se."driverId"
GROUP BY d."driverId", d.forename, d.surname
ORDER BY career_points DESC;
```

## 3.4 View 4: v_pitstop_efficiency

**Based on:** Query 4 – Pit-stop efficiency | **Purpose:** Pit-stop performance of each constructor across seasons

```
CREATE OR REPLACE VIEW "DBMS_project".v_pitstop_efficiency AS
WITH stops AS (                      -- attach the constructor and season to every stop
    SELECT ra.year, r."constructorId", ps.milliseconds
    FROM pit_stops ps
    JOIN results r ON r."raceId" = ps."raceId" AND r."driverId" = ps."driverId"
    JOIN races ra  ON ra."raceId" = ps."raceId"
    WHERE ps.milliseconds < 60000    -- exclude red-flag stoppages and long repairs
),
season_avg AS (
    SELECT year, "constructorId",
           COUNT(*)                        AS stops,
           AVG(milliseconds) / 1000.0      AS avg_stop_s,
           RANK() OVER (PARTITION BY year ORDER BY AVG(milliseconds)) AS season_rank
    FROM stops
    GROUP BY year, "constructorId"
    HAVING COUNT(*) >= 20            -- ignore teams with very few stops in a season
)
SELECT c.name                                      AS constructor,
       COUNT(*)                                    AS seasons,
       COUNT(*) FILTER (WHERE sa.season_rank = 1)   AS seasons_fastest,
       ROUND(AVG(sa.season_rank), 1)               AS avg_season_rank,
       ROUND(MIN(sa.avg_stop_s), 2)                AS best_season_avg_s,
       ROUND(AVG(sa.avg_stop_s), 2)                AS overall_avg_s
FROM season_avg sa
JOIN constructors c ON c."constructorId" = sa."constructorId"
GROUP BY c.name
HAVING COUNT(*) >= 3
ORDER BY seasons_fastest DESC, avg_season_rank;
```

## 3.5 View 5: v_dnf_analysis

**Based on:** Query 9 – DNF / retirement analysis | **Purpose:** Race retirements of each constructor grouped by cause

```
CREATE OR REPLACE VIEW "DBMS_project".v_dnf_analysis AS
WITH dnf AS (
    SELECT r."constructorId",
           s.status,
           CASE
             WHEN s.status ILIKE ANY (ARRAY['%engine%', '%power unit%', '%turbo%', '%overheating%', '%oil%',
                                            '%water%', '%radiator%', '%ers%', '%fuel%', '%ignition%',
                                            '%injection%', '%exhaust%', '%alternator%', '%magneto%', '%throttle%'])
                  THEN 'Engine / power unit'
             WHEN s.status ILIKE ANY (ARRAY['%accident%', '%collision%', '%spun off%', '%damage%'])
                  THEN 'Accident / collision'
             WHEN s.status ILIKE ANY (ARRAY['%gearbox%', '%transmission%', '%clutch%', '%driveshaft%',
                                            '%halfshaft%', '%differential%', '%axle%'])
                  THEN 'Gearbox / transmission'
             ELSE 'Other'
           END AS cause_group
    FROM results r
    JOIN status s ON s."statusId" = r."statusId"
    WHERE s.status <> 'Finished'
      AND s.status NOT LIKE '+%'                    -- '+1 Lap' etc. are classified finishers
      AND s.status NOT IN ('Did not qualify', 'Did not prequalify', 'Withdrew',
                           'Did not start', 'Disqualified', 'Excluded', '107% Rule')
)
SELECT c.name                                                          AS constructor,
       COUNT(*)                                                        AS dnfs,
       COUNT(*) FILTER (WHERE cause_group = 'Engine / power unit')     AS engine,
       COUNT(*) FILTER (WHERE cause_group = 'Accident / collision')    AS accident,
       COUNT(*) FILTER (WHERE cause_group = 'Gearbox / transmission')  AS gearbox,
       COUNT(*) FILTER (WHERE cause_group = 'Other')                   AS other,
       MODE() WITHIN GROUP (ORDER BY status)                           AS most_common_cause
FROM dnf
JOIN constructors c ON c."constructorId" = dnf."constructorId"
GROUP BY c.name
ORDER BY dnfs DESC;
```

## 3.6 View 6: v_season_points_progression_2021

**Based on:** Query 10 – Season points progression | **Purpose:** Race-by-race championship progression of the 2021 top 5 drivers

```
CREATE OR REPLACE VIEW "DBMS_project".v_season_points_progression_2021 AS
WITH season_races AS (
    SELECT "raceId", round, name
    FROM races
    WHERE year = 2021
),
top5 AS (                           -- top 5 of the final standings
    SELECT ds."driverId"
    FROM driver_standings ds
    WHERE ds."raceId" = (SELECT "raceId" FROM season_races ORDER BY round DESC LIMIT 1)
      AND ds.position <= 5
)
SELECT sr.round,
       sr.name                                                         AS race,
       d.surname                                                       AS driver,
       ds.points                                                       AS total_points,
       ds.points - COALESCE(LAG(ds.points, 1) OVER (PARTITION BY ds."driverId" ORDER BY sr.round), 0) AS points_this_race,
       MAX(ds.points) OVER (PARTITION BY sr.round) - ds.points         AS gap_to_leader,
       ds.position                                                     AS championship_position
FROM driver_standings ds
JOIN season_races sr ON sr."raceId" = ds."raceId"
JOIN top5 t          ON t."driverId" = ds."driverId"
JOIN drivers d       ON d."driverId" = ds."driverId"
ORDER BY sr.round, ds.position;
```

## 3.7 View 7: v_home_race_advantage

**Based on:** Query 12 – Home-race advantage | **Purpose:** Home Grand Prix finishing position compared with the driver's other races

```
CREATE OR REPLACE VIEW "DBMS_project".v_home_race_advantage AS
WITH nationality_country(nationality, country) AS (
    VALUES ('British','UK'), ('American','USA'), ('American','United States'), ('Italian','Italy'),
           ('French','France'), ('German','Germany'), ('Brazilian','Brazil'), ('Argentine','Argentina'),
           ('Belgian','Belgium'), ('South African','South Africa'), ('Swiss','Switzerland'),
           ('Japanese','Japan'), ('Australian','Australia'), ('Dutch','Netherlands'), ('Spanish','Spain'),
           ('Austrian','Austria'), ('Canadian','Canada'), ('Swedish','Sweden'), ('Mexican','Mexico'),
           ('Monegasque','Monaco'), ('Portuguese','Portugal'), ('Russian','Russia'), ('Chinese','China'),
           ('Indian','India'), ('Malaysian','Malaysia'), ('Hungarian','Hungary')
),
entries AS (
    SELECT r."driverId", ra.year, r."positionOrder",
           CASE WHEN nc.country IS NOT NULL THEN 1 ELSE 0 END AS is_home
    FROM results r
    JOIN races ra    ON ra."raceId" = r."raceId"
    JOIN circuits ci ON ci."circuitId" = ra."circuitId"
    JOIN drivers d   ON d."driverId" = r."driverId"
    LEFT JOIN nationality_country nc ON nc.nationality = d.nationality AND nc.country = ci.country
),
per_season AS (                      -- home races vs. the driver's other races in the same season
    SELECT "driverId", year,
           COUNT(*) FILTER (WHERE is_home = 1)               AS home_races,
           AVG("positionOrder") FILTER (WHERE is_home = 1)    AS avg_home,
           AVG("positionOrder") FILTER (WHERE is_home = 0)    AS avg_other
    FROM entries
    GROUP BY "driverId", year
    HAVING COUNT(*) FILTER (WHERE is_home = 1) > 0
       AND COUNT(*) FILTER (WHERE is_home = 0) > 0
)
SELECT d.forename || ' ' || d.surname               AS driver,
       d.nationality,
       SUM(ps.home_races)::int                     AS home_races,
       ROUND(AVG(ps.avg_home), 2)                  AS avg_finish_home,
       ROUND(AVG(ps.avg_other), 2)                 AS avg_finish_other,
       ROUND(AVG(ps.avg_other - ps.avg_home), 2)   AS places_gained_at_home,
       CASE WHEN AVG(ps.avg_other - ps.avg_home) > 0 THEN 'Better at home'
            ELSE 'Not better at home' END          AS verdict
FROM per_season ps
JOIN drivers d ON d."driverId" = ps."driverId"
GROUP BY d."driverId", d.forename, d.surname, d.nationality
HAVING SUM(ps.home_races) >= 8
ORDER BY home_races DESC;
```

## 3.8 View 8: v_decade_team_dominance

**Based on:** Query 13 – Decade-wise team dominance | **Purpose:** Top 3 constructors of each decade by share of points

```
CREATE OR REPLACE VIEW "DBMS_project".v_decade_team_dominance AS
WITH season_final_race AS (
    SELECT year, MAX(round) AS last_round
    FROM races
    GROUP BY year
),
season_points AS (                  -- constructors' final points of each season
    SELECT ra.year, cs."constructorId", cs.points
    FROM constructor_standings cs
    JOIN races ra             ON ra."raceId" = cs."raceId"
    JOIN season_final_race sf ON sf.year = ra.year AND sf.last_round = ra.round
),
decade_points AS (
    SELECT (year / 10) * 10 AS decade, "constructorId", SUM(points) AS points
    FROM season_points
    GROUP BY decade, "constructorId"
),
ranked AS (
    SELECT decade, "constructorId", points,
           ROUND((100.0 * points / SUM(points) OVER (PARTITION BY decade))::numeric, 1) AS pct_share,
           RANK() OVER (PARTITION BY decade ORDER BY points DESC)           AS rank_in_decade
    FROM decade_points
)
SELECT r.decade, r.rank_in_decade, c.name AS constructor, r.points, r.pct_share
FROM ranked r
JOIN constructors c ON c."constructorId" = r."constructorId"
WHERE r.rank_in_decade <= 3
ORDER BY r.decade, r.rank_in_decade;
```

## 3.9 View 9: v_grid_vs_points

**Based on:** Query 14 – Grid position vs. points scored | **Purpose:** Points, wins and podiums by starting grid position (2010 onward)

```
CREATE OR REPLACE VIEW "DBMS_project".v_grid_vs_points AS
SELECT CASE
         WHEN r.grid = 0              THEN 'Pit lane'
         WHEN r.grid = 1              THEN 'P1 (pole)'
         WHEN r.grid BETWEEN 2 AND 3  THEN 'P2-P3'
         WHEN r.grid BETWEEN 4 AND 6  THEN 'P4-P6'
         WHEN r.grid BETWEEN 7 AND 10 THEN 'P7-P10'
         WHEN r.grid BETWEEN 11 AND 15 THEN 'P11-P15'
         ELSE 'P16+'
       END                                                                  AS grid_bucket,
       COUNT(*)                                                             AS starts,
       ROUND(AVG(r.points)::numeric, 2)                                            AS avg_points,
       ROUND(100.0 * COUNT(*) FILTER (WHERE NULLIF(r.position, '\N')::int = 1)  / COUNT(*), 1)   AS win_pct,
       ROUND(100.0 * COUNT(*) FILTER (WHERE NULLIF(r.position, '\N')::int <= 3) / COUNT(*), 1)   AS podium_pct,
       ROUND(100.0 * COUNT(*) FILTER (WHERE r.points > 0)    / COUNT(*), 1)   AS points_finish_pct
FROM results r
JOIN races ra ON ra."raceId" = r."raceId"
WHERE ra.year >= 2010
GROUP BY grid_bucket
ORDER BY MIN(CASE WHEN r.grid = 0 THEN 99 ELSE r.grid END);
```

### 3.10 View 10: v_sprint_vs_gp

**Based on:** Query 15 – Sprint vs. Grand Prix performance | **Purpose:** Sprint result compared with the Grand Prix result on the same weekend

```
CREATE OR REPLACE VIEW "DBMS_project".v_sprint_vs_gp AS
SELECT d.forename || ' ' || d.surname                              AS driver,
       COUNT(*)                                                    AS sprint_weekends,
       ROUND(AVG(s."positionOrder"), 2)                            AS avg_sprint_finish,
       ROUND(AVG(r."positionOrder"), 2)                            AS avg_gp_finish,
       ROUND(AVG(r."positionOrder" - s."positionOrder"), 2)        AS gp_minus_sprint,
       SUM(s.points)                                               AS sprint_points,
       SUM(r.points)                                               AS gp_points_same_weekends,
       COUNT(*) FILTER (WHERE s."positionOrder" = r."positionOrder") AS same_position_both
FROM sprint_results s
JOIN results r ON r."raceId" = s."raceId" AND r."driverId" = s."driverId"
JOIN races ra  ON ra."raceId" = s."raceId"
JOIN drivers d ON d."driverId" = s."driverId"
GROUP BY d."driverId", d.forename, d.surname
HAVING COUNT(*) >= 5
ORDER BY sprint_points DESC;
```

# 4. Checking the Views

After the views are created, they can be listed and queried like tables:

```
-- List all views saved in the schema
SELECT table_name
FROM information_schema.views
WHERE table_schema = 'DBMS_project'
ORDER BY table_name;

-- Examples of using the views
SELECT * FROM "DBMS_project".v_driver_career_wins LIMIT 10;

SELECT constructor, championships, title_years
FROM "DBMS_project".v_constructor_championships
WHERE championships >= 5;

SELECT driver, home_races, places_gained_at_home
FROM "DBMS_project".v_home_race_advantage
WHERE verdict = 'Better at home';
```